

TECHNICAL REPORT

IoT / EMBEDDED SYSTEMS

Project 4: MQTT Communication & Two-Board Device Pairing (OOP)

Daily Technical Report — Day 11

Platform	ESP32 · Arduino Framework · PlatformIO
Subsystem	MQTT Symmetric Two-Topic Pairing — Two Identical IR Sensor + L298N Motor Boards
Broker	Public MQTT Broker (broker.emqx.io / iot.coreflux.cloud)
Prepared For	IOT Training Program

Contents

1. Daily Overview	3
1.1 Summary.....	3
1.2 Tasks Completed Today	3
2. Understanding MQTT — Protocol Deep Dive.....	4
2.1 What Is MQTT, In Plain Terms	4
2.2 The Publish / Subscribe Model	4
2.3 Core Vocabulary	4
2.4 Why MQTT Is Considered Lightweight	5
2.5 MQTT vs. ESP-NOW vs. HTTP — Applied Comparison	5
3. Learning Steps — First Hands-On Experiment	5
4. Project Architecture — Symmetric Two-Topic Pairing.....	9
4.1 Objective.....	9
4.2 Why Two Topics Instead of One	9
4.3 Roles & Required File Structure	9
4.4 What Was Built.....	10
5. Implementation Notes & Key Behaviors	10
5.1 Connection & Message Handling.....	11
5.2 Why the Topic Guard Is Necessary	11
5.3 Safety Watchdog	11
5.4 Mapping Messages to Motor & LED State	11
5.5 Clean-Code / PlatformIO Notes Carried Forward From Project 3	11
6. End-to-End Communication Flow	12
7. References	12

1. Daily Overview

1.1 Summary

This session extended the Project 3 motor-control platform by replacing the point-to-point ESP-NOW link with an MQTT publish/subscribe architecture, using the same conceptual "pairing" idea from the ESP-NOW exercise: two ESP32 boards, owned by two different students, communicate indirectly through a shared broker instead of a direct MAC-address link. Before writing any firmware, the pairing was first rehearsed manually using a web-based MQTT client to connect to a public broker and exchange a publish and a subscribe by hand, exactly as covered in the Day 11 notes. Only after that hands-on step did the two boards get programmed: Board A and Board B are built identically, each carrying its own IR obstacle sensor and its own L298N-driven DC motor, so that either student's sensor reading can control the other student's motor. A public MQTT broker (broker.emqx.io, with iot.coreflux.cloud as an alternative) replaces the private local broker used in early experiments, so the pairing works over the open internet rather than only on a local Wi-Fi network.

Unlike a one-way sensor-to-motor link, this pairing uses two separate topics so that both boards act as publisher and subscriber at the same time: each board publishes its own IR sensor reading on one topic and listens for its partner's IR sensor reading on the other topic, using that reading to control its own, locally-attached motor. The firmware keeps the modular, object-oriented structure established in Project 3: a dedicated MQTT class wraps the WiFiClient/PubSubClient stack, a config.h header centralizes every credential, pin, topic string, and timing constant, and the existing L298N class is reused unchanged for motor direction and speed control. A software watchdog — a simple time-based timeout — was added so the motor is forced to stop automatically if updates from the partner board stop arriving, protecting against a lost Wi-Fi link, a dead broker connection, or a crashed partner board.

1.2 Tasks Completed Today

- Reviewed MQTT protocol fundamentals in depth: client/broker roles, the publish/subscribe model, topics, payloads, and the underlying network stack.
- Completed the guided first experiment: connected a browser-based MQTT client to a public broker, subscribed to a topic by hand, and published a test message to observe the full publish → broker → subscriber flow before touching any firmware.
- Compared MQTT against the previously used ESP-NOW protocol and against HTTP, focusing on overhead, coupling, and scalability.
- Selected and configured a public MQTT broker reachable from both student boards without any local infrastructure.
- Redesigned the pairing so both boards act as publisher and subscriber simultaneously, using two distinct topics instead of one, mirroring the two-way potential of the original ESP-NOW pairing.
- Implemented a dedicated MQTT class that owns the underlying network client and exposes a small, clean API for connecting, publishing, and reaching the callback registration.
- Added a topic guard inside the incoming-message handler so each board reacts only to messages arriving on the topic it is actually subscribed to, ignoring anything else.
- Wrote the application logic for each board: the incoming-message handler stops the board's own motor when the partner's IR sensor reports a detected object, and drives it forward when the partner reports

a clear path, and a detection-timeout watchdog forces a safety stop if no message is received within two seconds.

- Verified message delivery, LED state changes, and motor response end-to-end through the public broker using both topics.

2. Understanding MQTT — Protocol Deep Dive

2.1 What Is MQTT, In Plain Terms

MQTT stands for Message Queuing Telemetry Transport. It is a messaging protocol built specifically for devices that have limited memory, limited processing power, limited battery life, and often an unreliable network connection — exactly the constraints of a microcontroller such as the ESP32. Rather than devices talking to each other directly, every MQTT device (called a client) connects to a central piece of software called a broker, and all communication passes through that broker. A sensor never sends data straight to a motor, a dashboard, or a phone; it only ever sends data to the broker, and the broker decides who else should receive it.

This is a fundamentally different mindset from the request/response style used by protocols such as HTTP. In HTTP, a client asks a server a question and waits for an answer. In MQTT, a client publishes a piece of information once, and the broker pushes it out automatically to anyone who has expressed interest — no one has to ask for it again and again. This is why MQTT is described as event-driven rather than request-driven, and it is exactly the behavior this pairing relies on: as soon as the IR sensor state changes, the update reaches the motor board automatically, with no polling required.

2.2 The Publish / Subscribe Model

MQTT organizes communication around three simple actions. Publishing means a client sends a message to a named address, called a topic. Subscribing means a client tells the broker "send me every future message sent to this topic." The broker's job is to receive every publish, look up who is currently subscribed to that exact topic, and forward the message to each of them. A topic is not a physical thing and does not need to be created in advance — it simply comes into existence the moment any client publishes to it or subscribes to it, and it disappears again once nobody is using it.

2.3 Core Vocabulary

Term	Meaning
Client	Any device or application connected to the broker — an ESP32 board, a phone, a laptop, a dashboard
Broker	The software service (not a router or piece of hardware) that receives every publish and forwards it to the correct subscribers
Publisher	A client that is currently sending data to a topic
Subscriber	A client that has asked to receive data from a topic
Topic	The named address a message is sent to, e.g. a slash-separated string acting like a mailbox name
Payload	The actual data carried inside the message — the content, as opposed to the address

2.4 Why MQTT Is Considered Lightweight

MQTT does not avoid the normal network stack — it still runs on top of Wi-Fi, IP, and TCP, and it can optionally add TLS encryption on top of that, exactly like most other internet protocols. What makes it lightweight is the size of the MQTT layer itself: its fixed header can be as small as two bytes, compared to an HTTP request which carries a method, a URL, a version string, a host, and a full set of headers before the actual data even begins. For a microcontroller sending a short reading many times per second, that difference in overhead is significant, both in bandwidth and in the processing needed to build and parse each message.

2.5 MQTT vs. ESP-NOW vs. HTTP — Applied Comparison

Aspect	ESP-NOW	HTTP	MQTT
Addressing	Direct MAC address of the peer board	URL / server address	Topic name only — no device addresses
Coupling	Sender must know receiver's MAC	Client must know server's address	Publisher and subscriber never know about each other
Communication style	Direct device-to-device	Request / response	Publish / subscribe, event-driven
Range / medium	Local Wi-Fi radio range only	Anywhere with network access	Anywhere with internet access, via a broker
Typical overhead	Very low, but local only	Higher (headers, method, version)	Very low, application layer only

3. Learning Steps — First Hands-On Experiment

Before either board was programmed, the publish/subscribe idea was rehearsed manually using a browser-based MQTT client, so that publishing and subscribing could be seen and understood as two completely independent actions, each one testable on its own, before any firmware logic was layered on top.

Step 1 — Open a Web MQTT Client

A browser-based MQTT client — MQTTX Web Client, available at mqttx.app/web-client — was opened and used purely as a manual publisher/subscriber, with no code involved at this stage — its only job was to connect to the same public broker the boards would later use.

Step 2 — Configure the Connection (in MQTTX Web Client)

- Broker address entered as the host to connect to (the public broker used throughout this exercise)
- Port and protocol set for a browser connection — browsers cannot open a raw TCP socket, so the web client connects over WebSocket (Secure), which carries MQTT inside it
- A client identifier was set (or left auto-generated) so the broker could tell this connection apart from any other client
- Username and password were left empty, since the public broker used for this classroom exercise does not require authentication

Step 3 — Connect

Clicking Connect performed the same handshake the firmware performs later: the client opened a network connection to the broker and the broker acknowledged it, after which the client's status changed from disconnected to connected.

Step 4 — Subscribe to a Topic

A new subscription was created for a chosen topic name. This told the broker: from now on, forward me anything published to this exact topic. At this point nothing had been published yet, so nothing was received — subscribing only registers interest, it does not fetch history.

Step 5 — Publish a Message

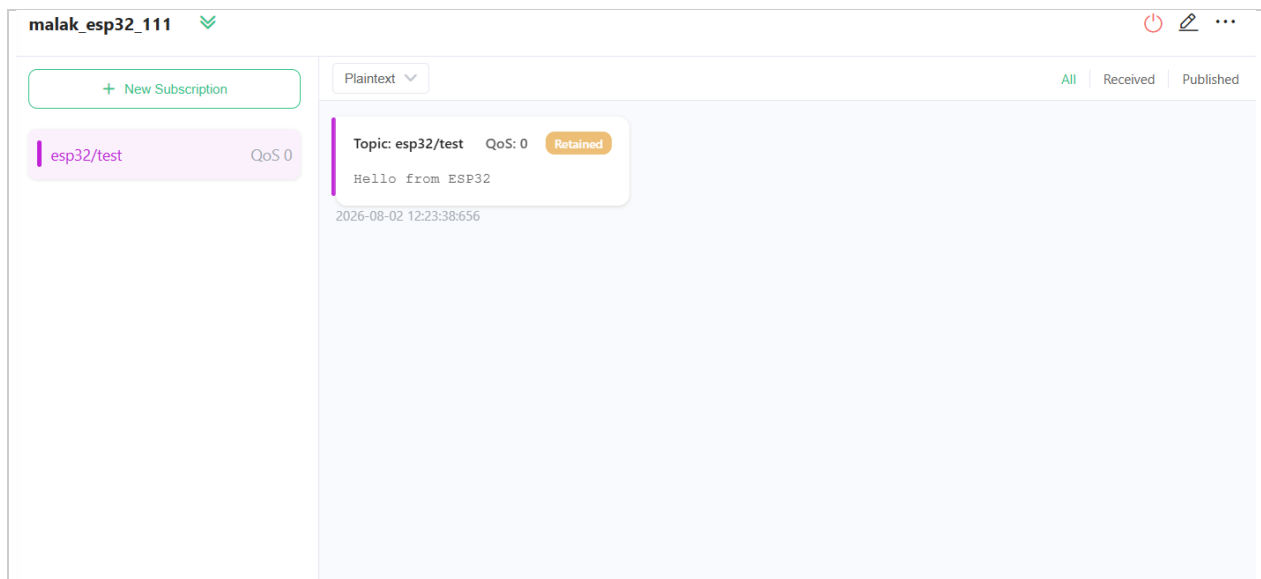
A short test message was published to that same topic name. The moment the message was sent, it appeared instantly in the subscription view, confirming the full loop: publish → broker → subscriber, with no direct connection between the two ever having existed.

Step 6 — Observe and Confirm Understanding

- The received message showed the topic, the payload, and the time it arrived — confirming the broker delivers the message, not just an acknowledgement
- Publishing to a different, unrelated topic while still subscribed to the original one produced no message, confirming that delivery is strictly topic-based
- Only after this manual exchange behaved as expected did the exercise move on to writing firmware that performs the same publish and subscribe actions automatically

Step 7 — First ESP32 Firmware Test (Publisher Role)

With the manual exchange confirmed, a small ESP32 sketch was flashed to replace the human publisher with real firmware. This is the exact sketch used to test against the MQTTX Web Client (mqttx.app/web-client): the board connected to Wi-Fi, connected to the same public broker (broker.emqx.io, port 8884) as an MQTT client identified as ESP32_1, and published a single message to the topic esp32/test as soon as the connection succeeded. The web client stayed subscribed to that same topic throughout, exactly as in Step 4, so it could pick up the firmware's publish the moment it arrived — no other configuration was changed between the manual test and this firmware test.



Result: the MQTTX Web Client (mqttx.app/web-client), connection "malak_esp32_111", still subscribed to esp32/test, receiving the message published by the ESP32 firmware — payload "Hello from ESP32", QoS 0, marked Retained, timestamped 2026-08-02 12:23:38.

- The topic, payload, and arrival time shown confirm the firmware reached the broker and the broker forwarded the message, closing the loop first rehearsed by hand in Steps 4–6
- The Retained badge indicates the broker stored this message as the last known value for esp32/test — any client subscribing to that topic afterwards receives it immediately, even without a new publish, which is a useful property for a status-style topic
- This confirms the ESP32 side of the pairing (connecting, publishing) before the receiving board's subscribe-and-react logic was layered on top in the full two-board project

Firmware Listing — ESP32 Publisher (Step 7)

The exact sketch flashed for this test is listed below, followed by a line-by-line breakdown. This is a minimal, publish-only sketch used purely to validate the connection and the publish path; it is not the final two-topic pairing firmware described in Section 4, which adds the MQTT class, config.h, subscribing, and the message-received handler.

```
#include <WiFi.h>
#include <PubSubClient.h>

const char* ssid = "CYBER_EXT";
const char* password = "cyberap2025";

const char* mqttServer = "broker.emqx.io";
const int mqttPort = 8884;

WiFiClient espClient;
PubSubClient client(espClient);

void setup()
{
  Serial.begin(115200);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED)
  {
    delay(500);
    Serial.print(".");
  }
  Serial.println("WiFi connected");
  client.setServer(mqttServer, mqttPort);
  if (client.connect("ESP32_1"))
  {
    Serial.println("MQTT Connected");
    client.publish("esp32/test", "Hello from ESP32");
  }
}

void loop()
{
  client.loop();
}
```

Line-by-Line Breakdown

- Library includes — `WiFi.h` provides the station-mode Wi-Fi connection used to reach the internet; `PubSubClient.h` provides the MQTT client itself (connect, publish, subscribe, loop). Neither library knows about the other: `PubSubClient` is built on top of whatever network client it is given, which is exactly what the next two lines set up.
- Wi-Fi credentials — the network name and password are stored as plain constants at the top of the file so the connection details are visible and easy to change in one place, the same central-settings habit carried into `config.h` in the full project.
- Broker configuration — the broker hostname and port are likewise stored as constants; `broker.emqx.io` on port 8884 is the same public broker and WebSocket-secure-style port used in the manual MQTTX web client test, which is why the two results can be compared directly.
- `WiFiClient espClient` — this object is the actual TCP network connection. On its own it has no idea what MQTT is; it only knows how to open a socket and move raw bytes.
- `PubSubClient client(espClient)` — the MQTT client is constructed on top of that network connection. This single line is the entire relationship between the network layer and the MQTT layer: `PubSubClient` uses `espClient` to actually send and receive bytes, and adds the MQTT protocol logic on top.
- `Serial.begin(115200)` — opens the serial monitor link used purely for debugging output; it has no effect on Wi-Fi or MQTT behavior.
- `WiFi.begin(ssid, password)` plus the while loop — starts the Wi-Fi connection attempt and then blocks, printing a dot roughly twice a second, until `WiFi.status()` reports `WL_CONNECTED`. This is a blocking wait: if the credentials are wrong or the network is unreachable, the board will sit in this loop indefinitely rather than continuing on.
- `client.setServer(mqttServer, mqttPort)` — this only records the broker address and port inside the `PubSubClient` object; it does not open a connection by itself, in the same way that saving a phone number is not the same as making the call.
- `client.connect("ESP32_1")` — this is the line that actually performs the MQTT handshake with the broker. The string "ESP32_1" is the MQTT client identifier the broker will use to recognize this connection; the call returns true only if the broker accepted the connection.
- `client.publish("esp32/test", "Hello from ESP32")` — runs only inside the `if(client.connect(...))` block, so the publish is only attempted after a successful connection; it sends one message, once, to the topic `esp32/test` with the payload "Hello from ESP32" — this is exactly the message captured in the screenshot above.
- `void loop() { client.loop(); }` — after `setup()` finishes, the board does not attempt to reconnect or publish again on its own; `loop()` only keeps calling `client.loop()`, which is required to service the underlying MQTT connection (keep-alive pings, incoming packet processing) but performs no new application-level action. This design is intentionally a one-shot publish test rather than a continuously repeating sensor publisher.

Design notes worth carrying forward

This sketch has no error message if `client.connect()` fails (the else branch is simply absent, so a failed MQTT connection fails silently after a successful Wi-Fi connection), no reconnect logic if the broker connection later drops, and no subscribe or callback at all, since this test only exercises the publish path. All three gaps were addressed in the full two-board project described in Sections 4–6: a dedicated MQTT class reports connection failures explicitly, the main loop can detect a dropped MQTT connection, and both boards register a callback to receive messages, not just send them.

Why this step matters

Rehearsing publish and subscribe manually, before any firmware exists, isolates protocol understanding from coding and debugging. Any confusion about "why didn't my message arrive" can be resolved at the level of broker, topic, and connection settings first, rather than mixed together with C++ logic later.

4. Project Architecture — Symmetric Two-Topic Pairing

4.1 Objective

Reproduce the ESP-NOW pairing exercise, but replace the direct ESP-NOW link between the two boards with MQTT publish/subscribe messaging over a public broker. The two boards belong to two different students, are built with identical hardware — each with its own IR obstacle sensor and its own L298N-driven DC motor — and must never address each other directly — all coordination happens exclusively through MQTT topics on the broker.

4.2 Why Two Topics Instead of One

Rather than giving one board a pure publisher role and the other a pure subscriber role, this pairing gives both boards the same capability: each board publishes its own IR sensor reading and subscribes to its partner's IR sensor reading, acting on that reading to control its own motor. This is achieved with two separate topic names — one topic carries messages in each direction. Every board runs the exact same MQTT class and the exact same overall firmware structure; only the two topic names configured in that board's settings file decide which topic it publishes to and which topic it subscribes to. Swapping those two topic names between the two boards is what creates the pairing, in the same way that the ESP-NOW exercise paired two boards by swapping which board held which partner's MAC address.

4.3 Roles & Required File Structure

Board / Student	Hardware	Publishes On	Subscribes To	Action Taken On Receive
Board A (Student 1)	IR obstacle sensor + L298N DC motor (identical to Board B)	Board A IR-status topic	Board B IR-status topic	Stops its own motor when Board B's IR sensor reports a detection; drives forward when Board B reports a clear path
Board B (Student 2)	IR obstacle sensor + L298N DC motor (identical to Board A)	Board B IR-status topic	Board A IR-status topic	Stops its own motor when Board A's IR sensor reports a detection; drives forward when Board A reports a clear path

Common file structure (both boards)

- A central settings file — Wi-Fi credentials, broker address and port, both topic names, and every pin definition
- An MQTT class (header + implementation) wrapping the underlying network client and MQTT library — connect, publish, subscribe, expose the callback hook
- An L298N class (header + implementation) — reused unchanged from Project 3 for motor direction/speed control
- A main application file — sets up the pins and the connection, then contains only the board-specific decision logic

4.4 What Was Built

1. Central Settings File

Exactly as in Project 3, every credential, pin number, and protocol constant lives in a single settings file so nothing is hard-coded inside an implementation file. This is what keeps the two boards interchangeable: the class code never changes between the two students' builds, only the settings file does — including which topic name is assigned to "publish" and which is assigned to "subscribe" on that particular board.

- Wi-Fi credentials
- Broker hostname and port
- Two topic name constants — one to publish on, one to subscribe to
- IR sensor pin, motor driver pins (direction and speed), status LED pins

2. MQTT Class

All MQTT plumbing is hidden behind a dedicated class instead of calling the underlying MQTT library directly from the main application file. The class owns the network client, connects to Wi-Fi, configures and connects to the broker, exposes a simple publish action, and exposes access to the underlying client object so the main application can register its own message-received handler and keep the connection alive on every loop.

3. L298N Motor Driver — Reused From Project 3

No changes were required to the motor driver class itself. It still exposes the same begin, forward, reverse, stop, and brake behavior; only the trigger for calling these actions changed — from an ESP-NOW receive event to an MQTT receive event.

4. Main Application Logic

Each board's main file stays intentionally thin: setup brings up the pins and the MQTT connection and registers the message handler; the main loop keeps the MQTT connection alive and runs the safety watchdog. On each board, all decision-making about its own motor state lives in exactly one place — the incoming-message handler — rather than being spread across the rest of the program.

5. Implementation Notes & Key Behaviors

The full firmware listing is maintained in the project repository and is not reproduced in this report; the notes below highlight the design decisions that matter most for understanding, grading, or debugging the pairing.

5.1 Connection & Message Handling

- Startup order matters: the pins are configured first, then the MQTT connection is established, and only once that succeeds is the message-received handler registered — so no message can be missed or mishandled during startup.
- If the connection attempt fails, the board reports the error clearly rather than continuing to run in an unknown, unconnected state.
- The connection's background processing must be given a chance to run on every pass through the main loop; skipping this would silently stop incoming packets from being received and keep-alive messages from being sent, even though the board looks otherwise fine.

5.2 Why the Topic Guard Is Necessary

Because both boards now subscribe to a topic and also publish on a different topic, the incoming-message handler checks which topic a given message actually arrived on before acting on it. This guard exists so that a board only reacts to messages meant for it — the one topic it is genuinely subscribed to — and ignores anything else, which becomes especially important once more than one topic, or more than one pair of boards, share the same broker.

5.3 Safety Watchdog

Each board keeps track of the last time it received a valid update from its partner. On every pass through the main loop, if too much time has passed since the last update — a short, fixed timeout — that board's own motor is force-stopped and its stop indicator is lit, regardless of whatever the last received command happened to be.

Why the watchdog matters

This protects each board's motor if the partner board loses Wi-Fi, the broker connection drops, or the partner board crashes: neither board simply keeps repeating the last known command forever. This is the same defensive pattern used for the E-Stop and overcurrent safety rule in Project 3, adapted to a networked link where silence itself now counts as a fault condition.

5.4 Mapping Messages to Motor & LED State

- A received value indicating the partner board's IR sensor detected an object turns the stop indicator on, turns the running indicator off, and stops the local motor.
- A received value indicating the partner board's path is clear turns the running indicator on, turns the stop indicator off, and drives the local motor forward at full speed.
- Any other, unrecognized value is currently ignored rather than acted on — a gap worth closing, noted in Section 7.

5.5 Clean-Code / PlatformIO Notes Carried Forward From Project 3

- Include guards at the top of every header file
- Clean separation maintained: declarations in header files, implementation in source files
- The central settings file is included wherever pin numbers, topics, or credentials are needed — never re-declared locally

- The main application file contains no direct calls into the underlying network or MQTT library — only calls through the MQTT class's own small API

6. End-to-End Communication Flow

The complete message path for one detection cycle, now using two topics so the exchange is symmetric in both directions:

- 1. Board A reads its IR sensor and evaluates whether an object is detected or the path is clear.
- 2. Board A publishes that state, as a simple value, to the Board A IR-status topic on the shared public broker.
- 3. The broker receives the publish and checks its subscription table for that exact topic.
- 4. The broker forwards the message to every client subscribed to the Board A IR-status topic — in this pairing, Board B.
- 5. Board B's message handler checks that the message truly arrived on the topic it is subscribed to, updates its last-received timestamp, and sets its own motor state and LEDs accordingly — stopping its motor if Board A reports a detection, or driving it forward if Board A reports a clear path.
- 6. Board B symmetrically publishes its own IR sensor reading on the Board B IR-status topic, which Board A is subscribed to; Board A reacts the same way, stopping or running its own motor based on Board B's reading — the same mechanism works in either direction.
- 7. If no new message arrives within the configured timeout — broker outage, Wi-Fi drop, or partner-board failure — the receiving board's watchdog forces a safety stop independently of the last received command.

Topic Design Summary

Topic	Published By	Subscribed By	Purpose
Board A IR-status topic	Board A (IR sensor + motor)	Board B (IR sensor + motor)	Reports Board A's detected/clear reading so Board B can stop or run its own motor accordingly
Board B IR-status topic	Board B (IR sensor + motor)	Board A (IR sensor + motor)	Reports Board B's detected/clear reading so Board A can stop or run its own motor accordingly

7. References

- [MQTT.org — official protocol](https://mqtt.org/)
- [GeeksforGeeks — Introduction of Message Queue Telemetry Transport Protocol \(MQTT\)](https://www.geeksforgeeks.org/mqtt-introduction/)
- <https://iotdunia.com/mqtt-protocol-complete-guide/>
- <https://www.emqx.com/en/blog/the-easiest-guide-to-getting-started-with-mqtt#mqtt-architecture-how-does-mqtt-work>

- <https://www.emqx.com/en/blog/the-ultimate-guide-to-mqtt-broker-comparison>
- <https://pcbsync.com/pubsubclient-mqtt-arduino/>
- <http://www.steves-internet-guide.com/using-arduino-pubsub-mqtt-client/>
- <https://scadaprotocols.com/mqtt-clients-brokers-connection/>
- <https://app.notion.com/p/Day-11-3b054ef921fa8014b409d4448b30d272>